

SCA TME 8 Réalisation d'une attaque par template

Delmotte Aurélien
Zhang Xiru

September 25, 2025

Contents

1	Introduction	2
2	Méthodologie	2
3	Implémentation	2
3.1	Construction des templates	2
3.2	Exécution de l'Attaque	5
4	Résultats	6
4.1	Efficacité	6
4.2	Contre-mesures	6
5	Conclusion	7

1 Introduction

Les attaques par template sont une méthode sophistiquée d'analyse de canaux auxiliaire, exploitant la consommation énergétique des dispositifs cryptographiques pour en déduire les clés secrètes. Cette technique repose sur deux phases principales : la construction de templates et leur utilisation pour attaquer un dispositif cryptographique.

2 Méthodologie

La construction des templates implique la collecte de traces de consommation énergétique, dénommées traces T0, lors de l'exécution du dispositif avec des clés et des plaintextes aléatoires. Chaque trace est analysée pour identifier des points d'intérêt (POIs) qui révèlent des informations potentielles sur les opérations cryptographiques internes, principalement autour de la sortie des SBox. Les données collectées sont ensuite utilisées pour établir des modèles statistiques, en supposant que la consommation pour chaque poids de Hamming de la valeur intermédiaire ciblée suit une distribution normale multivariée.

L'attaque proprement dite utilise ces templates pour évaluer et scorer des traces collectées sous une clé constante inconnue, appelées traces T1, en calculant la probabilité que les consommations mesurées correspondent aux différents modèles établis pendant la phase de construction. La clé avec le score cumulatif le plus élevé est identifiée comme la clé secrète du dispositif.

- T0 contient des clés et des plaintextes aléatoires.
- T1 contient une clé constante inconnue et des plaintextes associés.

3 Implémentation

Nous avons généré 10000 traces pour construire le template et 100 traces pour l'attaque, et sélectionné 5 points d'intérêt (POIs) pour l'analyse.

3.1 Construction des templates

Calcul des Points d'Intérêt (POIs) Cette fonction détermine les points dans la trace qui maximisent la variabilité de consommation selon les différentes valeurs de poids de Hamming. Elle utilise la fonction précédente pour évaluer chaque échantillon et sélectionner ceux qui offrent les variations les plus significatives, tout en évitant la sélection de points redondants à proximité immédiate des autres POIs sélectionnés.

```
1 void compute_poi(float_f **tmpl_traces, uint8_t **tmpl_textin, uint8_t **tmpl_key,
2                 int32_t poi_indexes[num_poi], int32_t nb_samples, int32_t byte) {
3     float_f diff_max[num_poi];
4     for(int i=0; i<num_poi; i++){
5         poi_indexes[i] = -poi_spacing;
6         diff_max[i] = 0;
7     }
8     for(int i=0; i<nb_samples; i++){
9         float_f a[NB_DISTRIB];
10        float_f p[NB_DISTRIB];
11        for(int j=0; j<NB_DISTRIB; j++) {p[j] = 0; a[j] = 0;}
12        for(int j=0; j<IMPL_NB_TRACES; j++){
13            int index;
14            #if USE_HW
15                index = hamming_weight(sbox[tmpl_textin[j][byte]^tmpl_key[j][byte]]);
16            #else
17                index = sbox[tmpl_textin[j][byte]^tmpl_key[j][byte]];
18            #endif
19            a[index]++;
20            p[index] += (tmpl_traces[j][i] - p[index]) / a[index];
21        }
22        float_f diff = compute_sum_abs_difference(p);
23        for(int j=0; j<NUM_POI; j++){
```

```

24         if(diff > diff_max[j]){
25             bool valid = true;
26             for(int k=0; k<NUM_POI; k++){
27                 if(k != j && mabs(i - poi_indexes[k]) <= poi_spacing){
28                     valid = false;
29                     break;
30                 }
31             }
32             if(valid){
33                 diff_max[j] = diff;
34                 poi_indexes[j] = i;
35             }
36         }
37     }
38 }
39 }

```

Calcul de la Somme des Différences Absolues Cette fonction qui est utilisée pour trouver les 5 POIs. Elle calcule la somme des valeurs absolues des différences entre toutes les paires d'éléments d'un tableau, où chaque élément correspond à la moyenne de consommation pour un poids de Hamming spécifique.

```

1 float_f compute_sum_abs_difference(float_f p[NB_DISTRIB]) {
2     float_f ret = 0;
3     for(int i=0; i<NB_DISTRIB; i++){
4         for(int j=i+1; j<NB_DISTRIB; j++){
5             ret += mabs(p[i] - p[j]);
6         }
7     }
8     return ret;
9 }

```

Calcul des paramètres statistiques Les fonctions suivantes sont implémentées pour calculer les paramètres statistiques essentiels dans la construction des templates pour une attaque par template. La première fonction, **compute_means**, calcule les moyennes de consommation à chaque point d'intérêt pour les différentes valeurs du poids de Hamming. La deuxième fonction, **compute_covariance**, évalue la covariance entre les points d'intérêt pour chaque distribution. Finalement, **compute_determinant_and_inv_covariance** calcule les déterminants et les inverses des matrices de covariance, qui sont essentiels pour la formulation de la distribution normale multivariée utilisée dans l'attaque.

```

1 void compute_means(float_f **tpl_traces, uint8_t **tpl_textin, uint8_t **tpl_key,
2     float_f mean[NB_DISTRIB][num_poi], int32_t poi_indexes[num_poi],
3     int32_t byte) {
4     for(int i = 0; i < num_poi; i++){
5         float_f a[NB_DISTRIB] = {0};
6         for(int j = 0; j < NB_DISTRIB; j++) {
7             mean[j][i] = 0;
8             a[j] = 0;
9         }
10        for(int j = 0; j < TMPL_NB_TRACES; j++){
11            int index = USE_HW ? hamming_weight(sbox[tpl_textin[j][byte] ^ tpl_key[j][
12                byte]]) :
13                sbox[tpl_textin[j][byte] ^ tpl_key[j][byte]];
14            a[index]++;
15            mean[index][i] += (tpl_traces[j][poi_indexes[i]] - mean[index][i]) / a[
16                index];
17        }
18    }
19 }
20 void compute_covariance(float_f **tpl_traces, uint8_t **tpl_textin, uint8_t **tpl_key
21     ,
22     float_f mean[NB_DISTRIB][num_poi], float_f covariance[NB_DISTRIB
23     ][num_poi][num_poi],

```

```

20         int32_t poi_indexes[num_poi], int32_t byte) {
21     for(int i = 0; i < num_poi; i++){
22         for(int j = i; j < num_poi; j++){
23             float_f sum[NB_DISTRIB] = {0};
24             int count[NB_DISTRIB] = {0};
25             for(int k = 0; k < TMPL_NB_TRACES; k++){
26                 int index = USE_HW ? hamming_weight(sbox[tmpl_textin[k][byte] ^ tmpl_key
[k][byte]]) :
27                     sbox[tmpl_textin[k][byte] ^ tmpl_key[k][byte]];
28                 float_f x_diff = tmpl_traces[k][poi_indexes[i]] - mean[index][i];
29                 float_f y_diff = tmpl_traces[k][poi_indexes[j]] - mean[index][j];
30                 sum[index] += x_diff * y_diff;
31                 count[index]++;
32             }
33             for(int k = 0; k < NB_DISTRIB; k++){
34                 if (count[k] > 1) {
35                     covariance[k][i][j] = sum[k] / (count[k] - 1.);
36                     covariance[k][j][i] = covariance[k][i][j];
37                 }
38             }
39         }
40     }
41 }
42
43 void compute_determinant_and_inv_covariance(float_f covariance[NB_DISTRIB][num_poi][
num_poi],
44         float_f determinant[NB_DISTRIB], float_f
inv_covariance[NB_DISTRIB][num_poi][num_poi]) {
45     for(int i = 0; i < NB_DISTRIB; i++){
46         determinant[i] = compute_determinant(covariance[i]);
47         compute_inverse(covariance[i], inv_covariance[i]);
48     }
49 }

```

Calcul des Templates Cette fonction coordonne le calcul des templates en orchestrant l'appel à plusieurs fonctions auxiliaires qui calculent les moyennes, covariances, déterminants, et inverses nécessaires pour les distributions. Elle sert de fonction principale pour l'assemblage des paramètres statistiques des templates.

```

1 void compute_template(float_f ** tmpl_traces, uint8_t ** tmpl_textin, uint8_t **
tmpl_key,
2         float_f mean[NB_DISTRIB][num_poi], float_f determinant[NB_DISTRIB
],
3         float_f inv_covariance[NB_DISTRIB][num_poi][num_poi], int32_t
poi_indexes[num_poi],
4         int32_t nb_samples, int32_t byte) {
5     compute_poi(tmpl_traces, tmpl_textin, tmpl_key, poi_indexes, nb_samples, byte);
6     compute_means(tmpl_traces, tmpl_textin, tmpl_key, mean, poi_indexes, byte);
7     float_f covariance[NB_DISTRIB][num_poi][num_poi];
8     compute_covariance(tmpl_traces, tmpl_textin, tmpl_key, mean, covariance, poi_indexes
, byte);
9     compute_determinant_and_inv_covariance(covariance, determinant, inv_covariance);
10 }

```

Fonction de Densité de Probabilité Multivariée Normale Cette fonction calcule la valeur de la distribution normale multivariée basée sur les moyennes, la matrice de covariance inverse, et le déterminant fournis. Elle évalue la probabilité d'un vecteur de consommation observé aux points d'intérêt, permettant de mesurer la correspondance entre les observations et le modèle statistique du template.

```

1 float_f multivariate_normal_pdf(float_f mean[num_poi], float_f determinant,
2         float_f inv_covariance[num_poi][num_poi], float_f
conso_vector[num_poi]) {
3     float_f truc = 0;
4     for(int i = 0; i < num_poi; i++){

```

```

5     float_f tmp = 0;
6     for(int j = 0; j < num_poi; j++){
7         tmp += (conso_vector[j] - mean[j]) * inv_covariance[i][j];
8     }
9     truc += tmp * (conso_vector[i] - mean[i]);
10 }
11 return (1. / (pow(2. * PI, num_poi / 2.) * pow(determinant, .5))) * exp(-.5 * truc);
12 }

```

3.2 Exécution de l'Attaque

Fonction d'Attaque La fonction `perform_attack` est conçue pour attaquer l'octet de la clé en exploitant les templates statistiques établis à partir de l'ensemble de traces T0. Cette attaque est effectuée sur l'ensemble de traces T1, qui contient les traces obtenues avec une clé constante inconnue. L'objectif est d'identifier l'octet de clé le plus probable en analysant chaque trace individuelle de T1 à l'aide des informations statistiques précédemment calculées. Cette méthode systématique permet d'évaluer chaque possible octet de clé par rapport à l'ensemble des traces T1, en utilisant des mesures de divergence basées sur la densité de probabilité multivariée normale.

```

1 uint8_t perform_attack(float_f ** rdev_traces, uint8_t ** rdev_textin, uint8_t *
   rdev_key,
2
3     float_f mean[NB_DISTRIB][num_poi], float_f determinant[NB_DISTRIB
   ],
4     float_f inv_covariance[NB_DISTRIB][num_poi][num_poi], int32_t
   poi_indexes[num_poi],
5     int32_t byte) {
6     // Initialisation des scores pour chaque possible octet de clé
7     float_f score[0x100];
8     for(uint16_t k = 0; k < 0x100; k++) score[k] = 1.0;
9     // Traitement de chaque trace pour évaluer les scores des clés potentielles
10    for(int i = 0; i < RDEV_NB_TRACES; i++){
11        // Extraction des caractéristiques à partir des points d'intérêt (POI)
12        float_f conso_vector[num_poi];
13        for(int j = 0; j < num_poi; j++) conso_vector[j] = rdev_traces[i][poi_indexes[j]
14    ]];
15        // Calcul des scores basé sur la distribution normale multivariée
16        float_f score9[NB_DISTRIB];
17        for(int j = 0; j < NB_DISTRIB; j++) {
18            score9[j] = multivariate_normal_pdf(mean[j], determinant[j], inv_covariance[
19    j], conso_vector);
20        }
21        // Accumulation des scores pour chaque octet de clé possible
22        for(uint16_t k = 0; k < 0x100; k++){
23            int index = USE_HW ? hamming_weight(sbox[rdev_textin[i][byte] ^ k]) :
24            sbox[rdev_textin[i][byte] ^ k];
25            score[k] += log(score9[index]); //logarithmic scoring for overflow prevention
26        }
27    }
28    // Détermination de l'octet de clé avec le score le plus élevé
29    float_f score_max = 0;
30    uint8_t kg = 0;
31    for(uint16_t k = 0; k < 0x100; k++){
32        if(score[k] > score_max) {
33            score_max = score[k];
34            kg = k;
35        }
36    }
37    return kg;
38 }

```

Dans cette fonction, chaque trace du dispositif réel est analysée pour calculer un score pour chaque valeur possible de l'octet de clé. Le score est calculé en utilisant la densité de probabilité normale multivariée, qui prend en compte les moyennes, les covariances inversées et les déterminants des distributions pour les différents poids de Hamming de l'octet attaqué. L'octet de clé

qui accumule le score le plus élevé à travers toutes les traces est considéré comme la clé probable.

4 Résultats

```

### Setup for Template part ###
# Num samples: 8200
# 3500 traces
# Trace file: traces_tmpl.raw
# Plaintext file: textin_tmpl.raw
# Key file: keylist_tmpl.raw
###
### Setup for Real Device part ###
# Num samples: 8200
# 20 traces
# Trace file: traces_rdev.raw
# Plaintext file: textin_rdev.raw
# Key file: key_rdev.raw
###
Correct Key:
# Key : [ 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c ]
Key Guess:
# Key : [ 0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6, 0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c ]
# Nb. bytes found: 16

```

Figure 1: Résultat de l'attaque démontrant la réussite de l'extraction de la clé

4.1 Efficacité

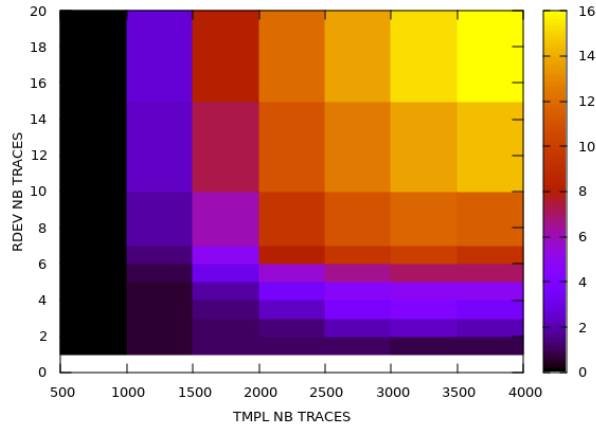


Figure 2: Résultat de l'attaque pour différents paramètres

L'attaque est efficace à partir de 15 traces T1 et 3500 traces T0. Et il est notable que plus de trames T0 (ou inversement T1) n'aide que marginalement si le nombre de trames T1 (ou inversement T0) est insuffisant.

4.2 Contre-mesures

L'AES Masqué tel que décrit dans le TME6 a été testé avec cette attaque, mais bien que l'attaque semble un peu moins efficace, il est toujours possible d'obtenir la clé secrète.

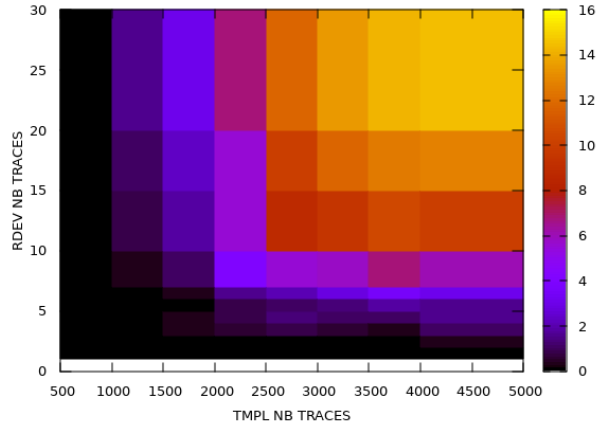


Figure 3: Résultat de l'attaque pour différents paramètres

L'attaque est efficace a partir de 30 traces T1 et 3500 traces T0.

5 Conclusion

Dans le cadre de cette étude, nous avons mis en œuvre une attaque par template sur des traces cryptographiques en utilisant une approche statistique détaillée. Nous avons développé et testé des fonctions spécifiques pour calculer les moyennes, les covariances, ainsi que les déterminants et les inverses des matrices de covariance, qui ont ensuite été utilisées pour construire des templates précis.

L'attaque a été réalisée sur un ensemble de traces T1, où chaque trace est associée à une clé constante et inconnue. Grâce aux templates établis, nous avons pu calculer avec succès les scores pour chaque octet de clé potentiel, en utilisant la probabilité de la distribution normale multivariée pour évaluer leur vraisemblance. L'octet de clé avec le score le plus élevé a été correctement identifié, démontrant ainsi l'efficacité de notre méthode d'attaque par template.

Une fois la template générée le nombre de traces nécessaire pour retrouver une clé secrète devient assez minime ce qui rends l'attaque assez efficace, on pourrais notamment imaginer des implémentations embarquées efficaces de l'attaque.